<!-- Extracted from https://oracc.museum.upenn.edu/doc/opendata/json/index.html on 2026-10-16 by a stdlib HTML->Markdown converter; see raw .html for the original. -->

Oracc JSON Data: A brief introduction for programmers

JSON

Oracc makes much of its data available in JSON format, and more JSON data is planned. All Oracc JSON files consist of a single object which have a "type" member and, in the case of project data, a "project" member. The data structure for each type is described here.

Top-level Oracc Data

Two files are provided at the top level: a simple list of public projects (projects.json) and a complex list of public projects analogous to the one-page listing of projects with blurbs (projectlist.json). In each case, these can be retrieved by prefixing them with an Oracc server name, e.g., http://oracc.org/projects.json or http://oracc.museum.upenn.edu/projects.json.

projects.json: type "projects"
: Provides a simple list of project names which can be concatenated to an Oracc server name to provide the base URL for retrieving additional JSON objects:

```
{
        "type": "projects",
        "public": [
                "aemw/alalakh/idrimi",
                "amgg",
                "armep",
                "arrim",
```

The `public` array gives the project names in a form suitable for concatenating to the `http://` URL for an Oracc server. You can find the names of JSON files available for a project in the manifest, for which see below.

projectlist.json: type "projectlist"
: Provides a version of the data used in the project list at http://oracc.org/projectlist.html:

```
{
        "type": "projectlist",
        "projects": [{
                "pathname": "rimanum",
                "name": "The House of Prisoners",
                "abbrev": "Rīm-Anum",
                "blurb": "Rīm-Anum, king of Uruk (ca. 1741–1739 BC) revolted aga
        } , {
```


The `projects` gives a list of objects, one per project. Each object contains a subset of the material in the project's configuration object (or `config.xml` within the Oracc installation. This information is also available from each project's metadata.json, so the project list is primarily a convenience for anyone wanting to provide a summary of the projects available.

Project-level Data

Oracc compiles project content into a collection of XML data structures which have additional annotation and linkage. The goal is to expose all of this data in JSON format.

manifest.json: type "manifest"
: For any given project name in `/projects.json` you can retrieve `/[PROJECT]/manifest.json` to discover which JSON formats are provided by the project. This gives a simple list of JSON files retrievable from a project. For the project `rimanum`, the URL `http://oracc.org/rimanum/manifest.json`, yields the following:

```
{
        "type": "manifest",
        "project": "rimanum",
        "files": [
                "corpus.json",
                "index-akk-x-oldbab.json",
                "index-cat.json",
                "index-lem.json",
...
        ],
        "everything": "json.zip"
}
```

For each entry in the array `files` you can retrieve, e.g., `http://oracc.org/rimanum/corpus.json`.

metadata.json: type "metadata"
: This provides several objects: "config"--the configuration info for the project; "witnesses"--only present if projects use composite texts, this provides information on which manuscripts are witnesses of the composites in the project; and "formats", a collection of lists indicating the presence of transliterations, transliterations and lemmatized data in the project.

```
{
        "type": "metadata",
        "config": {
                "pathname": "rimanum",
                "name": "The House of Prisoners",
                "abbrev": "RÄ«m-Anum",
                ...
        },
        "formats": {
                "atf": [ "P295625","P296047","P296277","P296278", ... ],
                "lem": [ "P295625","P296047","P296277","P296278","P296414", ..
                "tr-en": [ "P295625","P296047","P296277","P296278", ... ],
                "xtf": [ "P295625","P296047","P296277", ... ]
        }
}
```

catalogue.json: type "catalogue"
: Provides the project's catalogue:


```
{
        "type": "catalogue",
        "project": "rimanum",
        "members": {
                "P295625": {
                        "author": "Simmons, Stephen D.",
                        "collection": "J. Pierpont Morgan Library Collection,
                        "date_of_origin": "Rim-Anum.01.10.01",
                        "dates_referenced": "Rim-Anum.01.10.01",
                        "designation": "YOS 14, 341",
                        "genre": "Administrative",
                        "height": "40",
                        "language": "Sumerian",
                        ...
```


Although projects may have their own catalogue fields, all projects provide at least one of id_text or id_composite (some use a mix); designation; period; and provenience.

corpus.json: type "corpus"
: The JSON file corpus.json is another manifest file: it lists the individual text editions that are located in the folder `corpusjson/`:

```
{
  "type": "corpus",
  "project": "rimanum",
  "members": {
    "P295625": "corpusjson/P295625.json",
    "P296047": "corpusjson/P296047.json",
    "P296277": "corpusjson/P296277.json",
    "P296278": "corpusjson/P296278.json",
    "P296414": "corpusjson/P296414.json",
    "P297038": "corpusjson/P297038.json",
    "P311964": "corpusjson/P311964.json",
    "P368396": "corpusjson/P368396.json",
    "P368398": "corpusjson/P368398.json",
    "P372766": "corpusjson/P372766.json",
...
```


JSON for individual text editions: type "cdl"
:

Oracc text editions consist of two structures: one is the XML version of the user's transliteration. The other is entirely generated by Oracc and provides access to the divisions, content, and lemmatization of the text in a relatively simple nested tree format. In the Oracc world this format is called "XCL", or XML Chunks and Lemmas: the XCL tree has only three primary node types: `c`, a chunk of text which may be the whole text, a sentence or unit, a clause, a phrase or possibly others; `d`, a discontinuity, e.g., a line-break, a surface transition, damage to the content of the text; `l`, a lemma, the lemmatization of the text. The name of the array of children of any chunk node is called "cdl" based on these three members.

Discontinuities and lemmata have a "text" property which can be concatenated to create text fragments.

```
{
  "type": "cdl",
  "project": "rimanum",
  "textid": "P295625",
  "cdl": [
    {
      "node": "c",
      "type": "text",
      "id": "P295625.U0",
      "cdl": [
        {
          "node": "d",
          "subtype": "tablet",
          "type": "tablet",
          "ref": "P295625.x374.1",
          "label": "x374"
        },
        {
          "node": "d",
          "subtype": "obverse",
          "type": "obverse",
          "ref": "P295625.o.2",
          "label": "o"
        },
        {
          "node": "c",
          "type": "discourse",
          "subtype": "body",
          "id": "P295625.U1",
          "cdl": [
            {
              "node": "c",
              "type": "sentence",
              "id": "P295625.U2",
              "label": "o 1 - r 2",
              "cdl": [
                {
                  "node": "d",
```

```
      "type": "line-start",
      "ref": "P295625.3",
      "n": "1",
      "label": "o 1"
    },
    {
      "node": "l",
      "frag": "5(BAN₂)",
      "id": "P295625.l02b23",
      "ref": "P295625.3.1",
      "inst": "n",
      "f": {
        "lang": "akk-x-oldbab",
        "form": "5(BAN₂)",
        "pos": "n"
      }
    },
    {
      "node": "l",
      "frag": "6(SILA₃)",
      "id": "P295625.l02b24",
      "ref": "P295625.3.2",
      "inst": "n",
      "f": {
        "lang": "akk-x-oldbab",
        "form": "6(SILA₃)",
        "pos": "n"
      }
    },
    {
      "node": "l",
      "frag": "ZI₃.DA",
      "id": "P295625.l02b25",
      "ref": "P295625.3.3",
      "inst": "qēmu[flour]N",
      "sig": "@rimanum%akk-x-oldbab:ZI₃.DA=qēmu[flour//flour]N'N$qēm
      "f": {
        "lang": "akk-x-oldbab",
        "form": "ZI₃.DA",
        "cf": "qēmu",
```

```
        "gw": "flour",
        "sense": "flour",
        "norm": "qēmu",
        "pos": "N",
        "epos": "N"
      }
    },
    {
      "node": "d",
      "type": "line-start",
      "ref": "P295625.4",
      "n": "2",
      "label": "o 2"
    },
    {
      "node": "l",
      "frag": "⌜ŠUKU",
      "id": "P295625.l02b26",
      "ref": "P295625.4.1",
      "inst": "kurummat[food allocation]N",
      "sig": "@rimanum%akk-x-oldbab:ŠUKU=kurummatu[food allocation//
      "f": {
        "lang": "akk-x-oldbab",
        "form": "ŠUKU",
        "cf": "kurummatu",
        "gw": "food allocation",
        "sense": "food allocation",
        "norm": "kurummat",
        "pos": "N",
        "epos": "N"
      }
    },
    {
      "node": "l",
      "frag": "E₂⌝",
      "id": "P295625.l02b27",
      "ref": "P295625.4.2",
      "inst": "bīt[house]N",
      "sig": "@rimanum%akk-x-oldbab:E₂=bītu[house//house]N'N$bīt",
      "f": {
```

```
        "lang": "akk-x-oldbab",
        "form": "E₂",
        "cf": "bītu",
        "gw": "house",
        "sense": "house",
        "norm": "bīt",
        "pos": "N",
        "epos": "N"
      }
    },
    {
      "node": "l",
      "frag": "a-si-ri",
      "id": "P295625.l02b28",
      "ref": "P295625.4.3",
      "inst": "asīrī[prisoner of war]N",
      "sig": "@rimanum%akk-x-oldbab:a-si-ri=asīru[captive//prisoner
      "f": {
        "lang": "akk-x-oldbab",
        "form": "a-si-ri",
        "cf": "asīru",
        "gw": "captive",
        "sense": "prisoner of war",
        "norm": "asīrī",
        "pos": "N",
        "epos": "N"
      }
    },
    {
      "node": "d",
      "frag": "/",
      "type": "punct",
      "ref": "P295625.4.4",
      "lang": "akk-x-oldbab",
      "gdl_delim": " "
    },
    {
      "node": "l",
      "frag": "ZI.GA",
      "id": "P295625.l02b29",
```

```
                  "ref": "P295625.4.5",
                  "inst": "ṣītu[expenditure]N",
                  "sig": "@rimanum%akk-x-oldbab:ZI.GA=ṣītu[exit//expenditure]N'N
                  "f": {
                    "lang": "akk-x-oldbab",
                    "form": "ZI.GA",
                    "cf": "ṣītu",
                    "gw": "exit",
                    "sense": "expenditure",
                    "norm": "ṣītu",
                    "pos": "N",
                    "epos": "N"
                  }
                },
...
```


In the `l` nodes of the example above, the "sig" property is the string version of the lemmatization associated with a word. To reduce the size of the JSON files, the parsed versions of the sig properties are collected together in a separate object in the corpus, called "sigs":


```
"sigs": {
                "@rimanum%akk-x-oldbab:30-be-el-i₃-li₂=Sîn-bēl-ilī[00//00]PN'PN$
                        "form": "30-be-el-i₃-li₂",
                        "cf": "Sîn-bēl-ilī",
                        "gw": "00",
                        "sense": "00",
                        "pos": "PN",
                        "epos": "PN",
                        "norm": "Sîn-bēl-ilī"

                } ,
```


You can use the string "sig" as a key to look up the parsed form.

glossary-XXX.json: type "glossary"

:

The glossary files are named on a template which puts the language code in the place-holder XXX in the heading to the section. For language `akk` there is a glossary file `glossary-XXX.json` and so on; you can see which ones are provided by checking the project's manifest.

Glossaries are a list of entries which give the distributional data on all of the facets of Oracc lemmatization, gathered under a series of headings for spellings (`forms`); normalizations (`norms`) and meanings (`senses`. The same data is also given for the full signatures which reference the entry (`sigs`).

Several instance-related properties are common to many of these data:

icount
: The instance count for the datum.

ipct
: The percentage of instances of the datum that this count represents.

xis
: A reference to the compilation of instances that make up the count for the datum.

The second element of a glossary object is the set of xis data for the glossary: you can use the reference given in the xis property to access the list of word IDs which makes up the instance set to access the lemmatizations given in the corpus and traverse the context of any instance.

```
{
  "type": "glossary",
  "project": "rimanum",
  ...
  "lang": "akk-x-oldbab",
  "entries": [
    {
      "headword": "DUMU.EDUBA[(military) scribe]N",
      "id": "akk-x-oldbab.x000021",
      "icount": "7",
      "ipct": "100",
      "xis": "akk.r00000",
      "cf": "DUMU.EDUBA",
      "gw": "(military) scribe",
      "pos": "N",
      "forms": [
        {
          "type": "form",
          "id": "akk-x-oldbab.x000229",
          "n": "DUMU.E₂.DUB.BA",
          "icount": "4",
          "ipct": "57",
          "xis": "akk.r00001"
        },
         ...
      ],
      "norms": [
        {
          "id": "akk-x-oldbab.x000231",
          "icount": "7",
          "ipct": "100",
          "xis": "akk.r00000",
          "n": "DUMU.EDUBA",
          "forms": [
            {
              "type": "normform",
              "id": "akk-x-oldbab.x000232",
              "ref": "akk-x-oldbab.x000229",
```

```
            "icount": "4",
            "ipct": "57",
            "xis": "akk.r00001"
          },
           ...
        ]
      }
    ],
    "senses": [
      {
        "type": "sense",
        "id": "akk-x-oldbab.x000234",
        "n": "DUMU.EDUBA[(military) scribe//(military) scribe]N'N",
        "icount": "7",
        "ipct": "100",
        "xis": "akk.r00000",
        "pos": "N",
        "mng": "(military) scribe",
        "forms": [
          {
            "type": "form",
            "id": "akk-x-oldbab.x000235",
            "n": "%akk-x-oldbab:DUMU.E₂.DUB.BA",
            "icount": "4",
            "ipct": "57",
            "xis": "akk.r00001"
          },
           ...
        ],
        "norms": [
          {
            "id": "akk-x-oldbab.x000237",
            "n": "DUMU.EDUBA",
            "icount": "7",
            "ipct": "100",
            "xis": "akk.r00000"
          }
        ],
        "sigs": [
          {
```

```
              "type": "sig",
              "id": "akk-x-oldbab.x000238",
              "sig": "@rimanum%akk-x-oldbab:DUMU.E₂.DUB.BA.A=DUMU.EDUBA[(militar
              "icount": "3",
              "ipct": "43",
              "xis": "akk.r00002"
            },
             ...
          ]
        }
      ]
    },
    ...
  },
  "instances": {
    "akk.r0019f": [
      "rimanum:P405412.8.4"
    ],
    "akk.r0019b": [
      "rimanum:P405162.3.2",
      "rimanum:P405163.3.2",
      "rimanum:P405164.3.3",
      "rimanum:P405165.4.3",
      "rimanum:P405166.3.2",
      "rimanum:P405167.3.3"
    ],
    "akk.r001a1": [
      "rimanum:P372792.5.4",
      "rimanum:P405339.4.3",
      "rimanum:P405373.4.6",
      "rimanum:P405379.5.3"
    ],
    ...
```

index-xxx.json: type "index"

:

The index-xxx.json files are exports of a subset of the index data created and used by the Oracc search engine, giving the keys the indexer has generated from the input words and the locations in which they occur in the corpus. These keys may have been normalized using a variety of processes: accents are rendered as numeric indices; case may be foled; for English translation indexes a stemmer is used so that, e.g., "received" and "receives" will be gathered together under "receive" in the index.


```
{
	"type": "index",
	"project":"rimanum",
	"name": "cat",
	"keys": [{
		"key": "1",
		"count": "16",
		"instances": [
			"rimanum:P405220","rimanum:P405220","rimanum:P405246","r
		]},{
		"key": "3",
		"count": "16",
		"instances": [
			"rimanum:P405225","rimanum:P405225","rimanum:P405229","r
		]},{
		"key": "5",
		"count": "11",
		"instances": [
			"rimanum:P405212","rimanum:P405212","rimanum:P405250","r
		]},{
		"key": "6",
		"count": "11",
		"instances": [
			"rimanum:P405219","rimanum:P405219","rimanum:P405251","r
		]},{
```


The text IDs are always qualified with a project name because any project can use texts from other projects. For "txt", "lem" and "tra" index types, the instances are given as word IDs, so they can be used to locate the instance in the text edition. A simple way of displaying instances

is to use the URL `http://oracc.org/PROJECT/INSTANCE_ID/html`, e.g., `http://oracc.org/rimanum/P405219.4.1/html`. If you omit the "/html" the text is loaded into the Oracc pager instead of retrieving the simple HTML version.

18 Dec 2019 osc at oracc dot org

Steve Tinney & Eleanor Robson

Steve Tinney & Eleanor Robson, 'Oracc JSON Data: A brief introduction for programmers', *Oracc: The Open Richly Annotated Cuneiform Corpus*, Oracc, 2019 [http://oracc.museum.upenn.edu/doc/opendata/json/]

Back to top ^^

Released under a Creative Commons Attribution Share-Alike license 3.0, 2014.
[http://www.facebook.com/opencuneiform] [http://oracc.blogspot.com]
[http://www.twitter.com/oracctivity]

Oracc uses cookies only to collect Google Analytics data. Read more here; see the stats here [http://www.seethestats.com/site/oracc.museum.upenn.edu]; opt out here.